

# DataFrame vs RDD vs SQL

## DataFrame vs RDD

В этой секции мы сравниваем высокоуровневый DataFrame API с низкоуровневым RDD API.

### Результаты бенчмарков

| Кейс | Операция                    | DataFrame (ms) | RDD (ms) | Ускорение |
|------|-----------------------------|----------------|----------|-----------|
| 1    | Множественные агрегации     | 70.5           | 107.9    | 1.53x     |
| 2    | Оконные функции (Top-N)     | 78.4           | 181.9    | 2.32x     |
| 3    | Условная логика (Case/When) | 34.4           | 67.5     | 1.96x     |

### Анализ и объяснение оптимизаций

#### Множественные агрегации (`groupBy().agg(...)`)

- Код DataFrame: `df.groupBy("id").agg(sum, avg, min, max)`
- Код RDD: `rdd.map(...).reduceByKey(...)`
- Почему DataFrame быстрее:
  - **Catalyst Optimizer:** Объединяет все агрегатные функции в один физический оператор.
  - **Tungsten:** Использует `HashAggregate` с кодогенерацией (Whole-Stage Code Generation). Данные обрабатываются в бинарном формате (UnsafeRow) без десериализации в Python-объекты, что критично для PySpark.
  - **RDD:** Требуется сериализация данных из JVM в Python worker и обратно (Pickle), плюс ручная реализация `reduceByKey` менее эффективна, чем оптимизированный хеш-агрегат на C++/Java уровне.

#### Кейс 2: Оконные функции

- Код DataFrame: `Window.partitionBy(...).orderBy(...) + filter(rank <= 5)`
- Код RDD: `groupBy(...).mapValues(sorted(...)[:5])`
- Почему DataFrame быстрее:
  - **WindowExec:** Spark использует специализированный физический оператор для оконных функций.
  - **Память:** RDD `groupBy` (или `groupByKey`) требует сбора всех значений ключа в память (iterable), что может привести к OOM на больших группах. DataFrame может выполнять сортировку и ранжирование более эффективно, используя disk spill при необходимости.
  - **Сортировка:** Tungsten выполняет сортировку над бинарными данными (Radix Sort), что значительно быстрее сортировки Python-объектов.

#### Кейс 3: Условная логика (`when().otherwise()`)

- Код DataFrame: `F.when(...).otherwise(...)`
- Код RDD: `lambda row: ... if ... else ...`

- **Почему DataFrame быстрее:**

- **Проекция:** Catalyst компилирует цепочку `when/otherwise` в одно выражение `CaseWhen` внутри оператора `Project`.
- **Векторизация:** Вычисления происходят внутри JVM без переключения контекста в Python для каждой строки. В RDD лямбда-функция вызывается для *каждой* строки, создавая огромный оверхед на вызов функций и сериализацию.

---

## SQL vs DataFrame API

В этой секции мы сравниваем декларативный SQL с императивным построением запросов через DataFrame API.

### Результаты бенчмарков

| Кейс | Операция           | SQL (ms) | DataFrame (ms) | Разница                  |
|------|--------------------|----------|----------------|--------------------------|
| 1    | Сложная фильтрация | 82.7     | 77.4           | ~0.94x (DF чуть быстрее) |
| 2    | Строковые операции | 46.3     | 45.7           | ~1.00x (Одинаково)       |

### Анализ планов выполнения

#### Сложная фильтрация с подзапросом

В данном тесте DataFrame оказался немного быстрее SQL (77ms vs 82ms), хотя теоретически они должны быть близки.

- **SQL:** `WHERE bid_price > (SELECT avg(bid_price)...`
- **DataFrame:** Вычисляем `avg` отдельно через `collect()`, затем передаем значение в `filter()`.
- **Причина:** В подходе с DataFrame мы явно разделили вычисление среднего и фильтрацию. В SQL Spark строит план с подзапросом (SubqueryExec). Иногда явное вычисление скаляра (как мы сделали в DF) позволяет избежать накладных расходов на планирование и выполнение broadcast-join или subquery внутри Spark, особенно на малых данных. На больших данных SQL оптимизатор мог бы выиграть за счет более умного переиспользования сканирования (reuse exchange).

#### Строковые операции

Результаты практически идентичны (46.3ms vs 45.7ms).

- **Причина:** И SQL, и DataFrame API в конечном итоге транслируются в одни и те же логические и физические планы Catalyst.
  - `lower(col)` в SQL -> `Lower` expression в Catalyst.
  - `F.lower(col)` в DF -> `Lower` expression в Catalyst.
- Поскольку план одинаковый, производительность тоже одинаковая. Это подтверждает, что DataFrame API — это просто "DSL для построения SQL-планов".

## Выводы: Когда какой API выбирать?

### 1. Всегда предпочитайте DataFrame/SQL вместо RDD в PySpark.

- RDD в Python страдает от двойной сериализации (JVM <-> Python) и не использует оптимизации Tungsten/Catalyst.
- Выигрыш DataFrame может достигать 2х-10х и более на реальных объемах данных.

### 2. DataFrame vs SQL — дело вкуса и удобства.

- Для сложных цепочек преобразований, которые нужно параметризовать или переиспользовать, **DataFrame API** удобнее (программная композиция).
- Для аналитических запросов "ad-hoc" или при миграции с традиционных СУБД **SQL** проще и понятнее.
- Производительность в 99% случаев будет идентичной, так как под капотом работает один и тот же движок.

### 3. Исключения:

- Если логику невозможно выразить стандартными функциями Spark SQL (сложная математика, сторонние библиотеки), приходится использовать **UDF**. В этом случае лучше использовать **Pandas UDF** (Arrow), чем обычные Python UDF, чтобы минимизировать потери производительности.